

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Тема: Коммивояжёр
Вариант 1

Студент гр. 4345

Бабий Д.Н.

Преподаватель

Жангиров Т.Р.

Цель работы

Изучить и реализовать алгоритм Литтла, приближённый алгоритм: 2-приближение

Постановка задачи

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные:

n - количество городов ($5 \leq n \leq 15$).

Матрица расстояний между городами размером $n \times n$, где $graph[i][j]$ обозначает расстояние от города i до города j . Если $graph[i][j]=0$ (и $i \neq j$), это означает, что прямого пути между городами нет.

Выходные данные:

Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.

Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.

Если такого пути не существует, вывести "no path".

Разработайте программу, которая решает задачу коммивояжера при помощи 2-приближенного алгоритма. В данной постановке задачи нужно вернуться в исходную вершину после прохождения всех остальных вершин. При обходе остоного дерева (MST) необходимо идти по минимальному допустимому ребру из текущего. Каждая вершина в графе обозначается неотрицательным числом, начиная с 0, каждое ребро имеет неотрицательный вес. В графе нет рёбер из вершины в саму себя, в матрице весов на месте таких отсутствующих рёбер стоит значение -1.

Пример входных данных

2

-1 18.97 22.36 19.42 3.61

18.97 -1 35.61 38.01 17.0
22.36 35.61 -1 16.28 21.19
19.42 38.01 16.28 -1 21.02
3.61 17.0 21.19 21.02 -1

В первой строке указывается начальная вершина.

Далее идёт матрица весов.

В качестве выходных данных необходимо представить длину пути, полученного при помощи алгоритма. Следующей строкой необходимо представить путь, в котором перечислены вершины, по которым необходимо пройти от начальной вершины. Для приведённых в примере входных данных ответом будет

91.92

2 3 0 4 1 2.

Вариант 1. МВиГ: Алгоритм Литтла. Приближённый алгоритм: 2-приближение по МОД (АДО МОД).Замечание к варианту 1. АДО МОД является 2-приближением только для евклидовой матрицы. Начинать обход МОД со стартовой вершины.

Выполнение работы

1) Алгоритм Литтла

На вход в программу подаются размер квадратной матрицы N и сама матрица. Эта матрица является матрицей расстояний, где значение в $matrix[i][j]$ задаёт стоимость перехода из города i в город j . По главной диагонали значения принимают значения “бесконечности”, так как является петлей.

Реализованные функции:

1.1 `copy_matrix(matrix: list[list]) → list[list]`

Создаёт полную копию матрицы для независимой обработки ветвей дерева решений.

Пространственная сложность: $O(N^2)$

Временная сложность: $O(N^2)$

2.2 `reduce_matrix(matrix: list[list]) → list[list]`

Выполняется редукция строк и столбцов:

1. Для каждой строки находится минимальный элемент;
2. Минимум вычитается из всех элементов строки;
3. Аналогично выполняется редукция столбцов.

Сумма всех вычтенных минимумов образует нижнюю границу стоимости текущего решения.

Пространственная сложность: $O(N^2)$

Временная сложность: $O(N^2)$

2.3 `find_best_zero(matrix: list[list]) → list[list]`

Алгоритм выбирает нулевой элемент матрицы с максимальным штрафом (penalty). Для каждого из них вычисляется минимальный элемент 1) в строке без учёта текущего столбца; 2) в столбце без учёта текущей строки.

Штраф определяется как: $penalty = row_min + col_min$

Выбирается нулевой элемент с максимальным значением штрафа.

Пространственная сложность: $O(N)$

Временная сложность: $O(N^3)$

2.4 `creates_cycle(edges: list, start: int, end: int, n: int) → list[list]`

Функция проверяет, приведёт ли добавление нового ребра (start, end) к образованию преждевременного цикла.

Для этого строится временный ориентированный граф из уже выбранных рёбер, после чего выполняется проход по вершинам начиная с end. Если в процессе обхода происходит возврат в вершину start раньше, чем будут включены все n городов, значит образуется недопустимый подцикл, и функция возвращает True.

Таким образом предотвращается появление частичных циклов до построения полного гамильтонова цикла.

Пространственная сложность: $O(N)$

Временная сложность: $O(N)$

2.5 `little(matrix)`

Основная функция реализации алгоритма Литтла для решения задачи коммивояжёра методом ветвей и границ.

Алгоритм работает следующим образом:

1. Создаётся копия исходной матрицы расстояний;
2. Выполняется редукция матрицы для получения начальной нижней границы;
3. Формируется стек состояний дерева решений;
4. На каждом шаге:
 - выбирается нулевой элемент с максимальным штрафом;
 - создаются две ветви: исключение ребра и включение ребра в маршрут;
5. Для ветви включения:
 - запрещаются переходы из выбранной строки и в выбранный столбец;
 - запрещается обратное ребро;
 - проверяется образование подцикла;
 - выполняется повторная редукция матрицы;
6. Для ветви исключения:

- выбранное ребро помечается как недоступное;
 - матрица повторно редуцируется;
7. Алгоритм продолжает перебор состояний, пока стек не станет пустым;
 8. В результате выбирается маршрут с минимальной стоимостью.

Использование нижней границы позволяет отсекать невыгодные ветви и существенно уменьшать количество перебираемых состояний по сравнению с полным перебором.

Пространственная сложность: $O(N^2)$ для хранения матриц состояний.

Временная сложность:

в худшем случае — $O(N!)$, так как задача коммивояжёра является NP-трудной.

2.6 build_path(edges: list) → list

Функция восстанавливает итоговый маршрут коммивояжёра из списка выбранных рёбер.

Результатом работы функции является список вершин в порядке обхода маршрута.

Пространственная сложность: $O(N)$

Временная сложность: $O(N)$

Таблица 1. Тест алгоритма Литтла

№	Входные данные	Выходные данные	Комментарий
1	4 -1 7 3 8 5 -1 6 2 9 1 -1 4 3 8 5 -1	9.0 0 2 1 3 0	Полный граф из 4 городов, проверка корректного поиска минимального цикла
2	5 -1 14 9 7 20 6 -1 12 11 8 10 5 -1 13 7 9 6 4 -1 15 8 17 3 6 -1	32.0 0 3 2 1 4 0	Асимметричный граф с несколькими выгодными маршрутами
3	3	7.0	Проверка выбора короткого пути вместо

№	Входные данные	Выходные данные	Комментарий
	-1 50 2 1 -1 40 3 4 -1	0 2 1 0	дорогих переходов
4	5 -1 10 8 -1 6 7 -1 5 9 -1 4 3 -1 2 8 -1 6 1 -1 7 5 -1 9 4 -1	21.0 0 4 3 2 1 0	Проверка работы алгоритма при наличии недоступных рёбер (-1)

Исходный код программы приведён в приложении А.

2) 2-приближение по МОД

В данной работе реализован приближённый алгоритм на основе минимального остовного дерева (МОД). Алгоритм гарантирует получение решения, стоимость которого не превышает удвоенную стоимость оптимального маршрута для метрической задачи коммивояжёра.

На вход программе подаются стартовая вершина `start`, матрица расстояний `matrix`.

2.1 `build_mst(n: int, matrix: list[list]) → list[list]`

Функция строит минимальное остовное дерево графа с помощью алгоритма Прима.

Алгоритм работы:

1. Создаётся множество посещённых вершин;
2. Для каждой вершины хранится минимальная стоимость подключения к дереву;
3. На каждом шаге выбирается непосещённая вершина с минимальным весом;
4. Выполняется обновление границ для соседних вершин;
5. Формируется список родителей `parent`, описывающий структуру дерева;

6. На основе массива родителей создаётся список смежности дерева. После построения дерева списки соседей сортируются по весу рёбер.

Пространственная сложность: $O(N)$

Временная сложность: $O(N^2)$

2.2 dfs(tree: list[list]) → tuple

Функция выполняет обход минимального остовного дерева в глубину. Обход начинается со стартовой вершины, после чего каждая посещённая вершина добавляется в маршрут. Далее алгоритм рекурсивно переходит ко всем непосещённым соседним вершинам дерева, продолжая обход, пока не будут посещены все вершины графа. После завершения выполняется возврат в стартовую вершину (замкнутый цикл). Затем производится вычисление полной стоимости маршрута путём суммирования весов всех переходов пути. Полученный маршрут является приближённым решением задачи коммивояжёра, построенным на основе минимального остовного дерева.

Пространственная сложность: $O(N)$

Временная сложность: $O(N)$

Таблица 2. Тест приведенного

№	Входные данные	Выходные данные	Комментарий
0	-1 4 7 3		
1	4 -1 6 5 7 6 -1 2 3 5 2 -1	15.00 0 3 2 1 0	Полный симметричный граф из 4 вершин, базовая проверка построения МОД и DFS-обхода
2	1 -1 8 5 9 7	19.00 1 4 3 2 0 1	Проверка выбора рёбер минимального остовного дерева

№	Входные данные	Выходные данные	Комментарий
5	8 -1 6 4 3	24.00 3 2 1 4 0 3	при старте из вершины 1
	5 6 -1 2 8		
	9 4 2 -1 1		
	7 3 8 1 -1		
	3		
	-1 9 4 7 6		Граф с несколькими близкими по весу рёбрами, проверка порядка обхода соседей
	9 -1 2 5 8		
	4 2 -1 1 3		
	7 5 1 -1 4		
	6 8 3 4 -1		

Выводы

В работе реализованы две версии алгоритма Aho–Corasick algorithm для строкового поиска. Первая версия выполняет поиск множества шаблонов в тексте за один проход, используя бор, суффиксные и терминальные ссылки, и находит все вхождения без повторных сравнений. Вторая версия расширяет алгоритм для поддержки wildcard-символов: шаблон разбивается на блоки, которые ищутся отдельно, а затем результаты объединяются через массив совпадений. Обе реализации работают за линейное время и эффективно обрабатывают большие строки без избыточных вычислений.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл `ado_mod.py`

```
def print_matrix(matrix, title="Матрица"):
    print(f"\n{title}")

    for row in matrix:
        print(" ".join("INF" if x == INF else f"{int(x):3}" for x in
row))

def print_tree(tree):
    print("\nТекущее MST:")

    for i in range(len(tree)):
        print(f"{i}: {tree[i]}")

def build_mst(n, matrix):
    visited = [False] * n
    min_bound = [INF] * n
    parent = [-1] * n

    min_bound[start] = 0

    print("\n===== ПОСТРОЕНИЕ MST =====")

    for step in range(n):
        print(f"\n--- ШАГ {step + 1} ---")

        v = -1
        best = INF

        # Поиск вершины с минимальной границей
        for i in range(n):
            if not visited[i] and min_bound[i] < best:
                best = min_bound[i]
```

```

        v = i

    print(f"Выбрана вершина: {v}")
    print(f"Минимальная стоимость подключения: {best}")

    visited[v] = True

    print(f"Посещенные вершины: {visited}")

    # Обновление границ
    for to in range(n):
        if not visited[to] and matrix[v][to] < min_bound[to]:

            old = min_bound[to]

            min_bound[to] = matrix[v][to]
            parent[to] = v

            print(
                f"Обновление вершины {to}: "
                f"{old} -> {min_bound[to]} "
                f"(родитель {v})"
            )

    print(f"min_bound: {min_bound}")
    print(f"parent: {parent}")

# Построение дерева
tree = [[] for _ in range(n)]

print("\n===== ПОСТРОЕНИЕ ДЕРЕВА =====")

for i in range(n):
    if parent[i] != -1:
        p = parent[i]

        tree[p].append(i)
        tree[i].append(p)

```

```

        print(f"Ребро MST: {p} <-> {i}")

    for i in range(n):
        tree[i].sort(key=lambda x: matrix[i][x])

    print_tree(tree)

    return tree

def dfs(tree):
    visited = [False] * n
    path = []

    print("\n===== DFS ОБХОД =====")

    def walk(v):
        print(f"\nПереход в вершину {v}")

        visited[v] = True
        path.append(v)

        print(f"Текущий путь: {path}")

        for to in tree[v]:
            if not visited[to]:
                print(f"Из {v} идем в {to}")
                walk(to)

    walk(start)

    path.append(start)

    print(f"\nВозврат в стартовую вершину {start}")
    print(f"Итоговый путь: {path}")

    cost = 0.0

    print("\n===== ПОДСЧЕТ СТОИМОСТИ =====")

```

```

    for i in range(len(path) - 1):
        a = path[i]
        b = path[i + 1]

        edge_cost = matrix[a][b]

        print(f"{a} -> {b} = {edge_cost}")

        cost += edge_cost

    print(f"\nПолная стоимость: {cost}")

    return path, cost

start = int(input())

matrix = []

while True:
    try:
        line = input()

        if not line:
            break

        matrix.append(list(map(float, line.split())))

    except:
        break

n = len(matrix)
INF = float("inf")

for i in range(n):
    for j in range(n):
        if matrix[i][j] == -1:
            matrix[i][j] = INF

```

```

for i in range(n):
    matrix[i][i] = INF

print_matrix(matrix, "Исходная матрица")

tree = build_mst(n, matrix)

path, cost = dfs(tree)

print("\n===== РЕЗУЛЬТАТ =====")
print(f"Стоимость: {cost:.2f}")
print("Маршрут:", *path)

Файл ac_ws.py
def copy_matrix(matrix):
    return [row[:] for row in matrix]

def print_matrix(matrix, title="Матрица"):
    print(f"\n{title}")

    for row in matrix:
        print(" ".join("INF" if x == INF else f"{int(x):3}" for x in
row))

def reduce_matrix(matrix):
    n = len(matrix)
    reduction_cost = 0

    print_matrix(matrix, "До редукции")

    # Редукция строк
    for i in range(n):
        row_min = min(matrix[i])

        if row_min != INF and row_min > 0:
            print(f"Минимум строки {i}: {row_min}")

```

```

        reduction_cost += row_min

    for j in range(n):
        if matrix[i][j] != INF:
            matrix[i][j] -= row_min

    print_matrix(matrix, "После редукции строк")

# Редукция столбцов
for j in range(n):
    col_min = min(matrix[i][j] for i in range(n))

    if col_min != INF and col_min > 0:
        print(f"Минимум столбца {j}: {col_min}")

        reduction_cost += col_min

        for i in range(n):
            if matrix[i][j] != INF:
                matrix[i][j] -= col_min

    print_matrix(matrix, "После редукции столбцов")

    print(f"Стоимость редукции: {reduction_cost}\n")

return reduction_cost

def find_best_zero(matrix):
    n = len(matrix)

    best_penalty = -1
    best_i = -1
    best_j = -1

    print("\nОценки нулевых элементов:")

    for i in range(n):

```

```

    for j in range(n):
        if matrix[i][j] != 0:
            continue

        row_values = [
            matrix[i][k] for k in range(n) if k != j and matrix[i]
[k] != INF
        ]

        col_values = [
            matrix[k][j] for k in range(n) if k != i and matrix[k]
[j] != INF
        ]

        row_min = min(row_values) if row_values else 0
        col_min = min(col_values) if col_values else 0

        penalty = row_min + col_min

        print(
            f"Ноль ({i}, {j}) -> "
            f"min_row={row_min}, "
            f"min_col={col_min}, "
            f"штраф={penalty}"
        )

        if penalty > best_penalty:
            best_penalty = penalty
            best_i = i
            best_j = j

    print(f"\nВыбран ноль: ({best_i}, {best_j}) " f"со штрафом
{best_penalty}")

    return best_i, best_j

```

```

def creates_cycle(edges, start, end, n):
    graph = {}

```

```

for a, b in edges:
    graph[a] = b

graph[start] = end

current = end
length = 1

while current in graph:
    current = graph[current]
    length += 1

    if current == start:
        return length < n

return False

def little(matrix):
    n = len(matrix)

    original = copy_matrix(matrix)

    root_matrix = copy_matrix(matrix)

    print("\n===== НАЧАЛО АЛГОРИТМА ЛИТТЛА =====")

    lower_bound = reduce_matrix(root_matrix)

    print(f"Начальная нижняя граница: {lower_bound}")

    stack = [(root_matrix, lower_bound, [], 0)]

    best_cost = INF
    best_edges = []

    step = 1

```

```

while stack:
    print(f"\n\n===== ШАГ {step} =====")
    step += 1

    matrix_now, bound, edges, real_cost = stack.pop()

    print(f"Текущая граница: {bound}")
    print(f"Текущая стоимость: {real_cost}")
    print(f"Текущие ребра: {edges}")

    print_matrix(matrix_now, "Текущая матрица")

    if bound >= best_cost:
        print("Ветка отсечена")
        continue

    if len(edges) == n:
        print("Найден полный маршрут")

        if real_cost < best_cost:
            best_cost = real_cost
            best_edges = edges

            print(f"Новый лучший маршрут: {best_edges}")
            print(f"Новая лучшая стоимость: {best_cost}")

            continue

    i, j = find_best_zero(matrix_now)

    if i == -1:
        print("Нулей не найдено")

        used_from = {a for a, b in edges}
        used_to = {b for a, b in edges}

        remaining_from = [v for v in range(n) if v not in
used_from]
        remaining_to = [v for v in range(n) if v not in used_to]

```

```

if len(remaining_from) == 1 and len(remaining_to) == 1:
    a = remaining_from[0]
    b = remaining_to[0]

    if not creates_cycle(edges, a, b, n) or len(edges) ==
n - 1:

        final_edges = edges + [(a, b)]

        final_cost = real_cost + original[a][b]

        print(f"Автодобавление последнего ребра ({a},
{b})")

        if final_cost < best_cost:
            best_cost = final_cost
            best_edges = final_edges

            print(f"Новый лучший маршрут: {best_edges}")
            print(f"Новая лучшая стоимость: {best_cost}")

        continue

if not creates_cycle(edges, i, j, n):
    print(f"\n--- ВКЛЮЧАЕМ ребро ({i}, {j}) ---")

    include_matrix = copy_matrix(matrix_now)

    for k in range(n):
        include_matrix[i][k] = INF

    for k in range(n):
        include_matrix[k][j] = INF

    include_matrix[j][i] = INF

    reduction = reduce_matrix(include_matrix)

    new_real_cost = real_cost + original[i][j]

```

```

new_bound = bound + original[i][j] + reduction

print(f"Новая стоимость: {new_real_cost}")
print(f"Новая граница: {new_bound}")

stack.append(
    (
        include_matrix,
        new_bound,
        edges + [(i, j)],
        new_real_cost,
    )
)

else:
    print(f"Ребро ({i}, {j}) создает цикл")

# Исключение ребра
print(f"\n--- ИСКЛЮЧАЕМ ребро ({i}, {j}) ---")

exclude_matrix = copy_matrix(matrix_now)

exclude_matrix[i][j] = INF

reduction = reduce_matrix(exclude_matrix)

exclude_bound = real_cost + reduction

print(f"Граница после исключения: {exclude_bound}")

stack.append(
    (
        exclude_matrix,
        exclude_bound,
        edges,
        real_cost,
    )
)

```

```

    return best_cost, best_edges

def build_path(edges):
    graph = {}

    for a, b in edges:
        graph[a] = b

    path = [0]

    current = 0

    while True:
        current = graph[current]

        path.append(current)

        if current == 0:
            break

    return path

n = int(input())
INF = float("inf")
matrix = []

for _ in range(n):
    row = list(map(float, input().split()))

    matrix.append([INF if x == -1 else x for x in row])

cost, edges = little(matrix)

path = build_path(edges)

print("\n===== РЕЗУЛЬТАТ =====")
print("Минимальная стоимость:", cost)

```

```
print("Маршрут:", *path)
```